Specification-Driven Development

Why Specifications Become the Source of Truth in AI-Assisted Engineering

Amritanshu Singh

Contents

1 Introduction **2**

2 What is Specification-Driven Development? **2**

3 Development Workflow **2**

4 Why Specifications Become the Source of Truth **3**
4.1 Example 3

5 Prompt vs Specification **4**

6 Architecture of AI-Assisted Engineering **4**

7 Anatomy of a Good Specification **4**

8 Intent **5**

9 Scope **5**

10 Constraints **6**

11 Interfaces **6**

12 Acceptance Criteria **6**

13 Test Cases **7**

14 Conclusion **7**

1 Introduction

Software development has always depended on clear communication. Traditionally, business stakeholders describe a problem, analysts define requirements, developers write code, and testers verify whether the solution meets expectations. With the rise of Artificial Intelligence, many of these activities can now be assisted by AI systems that generate code, documentation, test cases, and even architecture diagrams.

Although AI has significantly improved productivity, it has also introduced a new challenge. AI systems depend heavily on the instructions they receive. If those instructions are incomplete or ambiguous, the generated output may not match the actual business need. Different developers may write different prompts for the same requirement and receive different results. This inconsistency can lead to confusion, rework, and increased project risk.

To overcome this challenge, organizations are increasingly adopting Specification-Driven Development (SDD). Instead of treating prompts as the foundation of development, SDD places a formal specification at the center of the process. The specification clearly defines what the system should do, what limitations exist, and how success will be measured. This creates a common understanding for all stakeholders, including developers, testers, and AI systems.

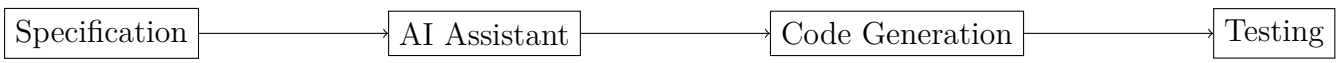
2 What is Specification-Driven Development?

Specification-Driven Development is a software development approach in which software is built from a detailed specification rather than from informal conversations or loosely defined requirements. A specification acts as a blueprint that describes the expected behavior of a system before development begins.

An easy way to understand this concept is to compare software development with constructing a building. Before construction starts, architects create detailed plans that describe the structure, dimensions, materials, and safety requirements of the building. Builders do not rely on verbal instructions alone because different people may interpret those instructions differently.

In the same way, software teams use specifications to ensure that everyone shares the same understanding of the product. By defining requirements in a structured manner, the specification reduces ambiguity and provides a reliable foundation for development activities. When AI systems are involved, the specification becomes especially important because it guides the AI toward generating outputs that align with business goals.

3 Development Workflow



In this workflow, the specification serves as the foundation. The AI uses the specification to generate code and related artifacts, while testing ensures that the generated solution satisfies the documented requirements.

4 Why Specifications Become the Source of Truth

When developers first begin using AI tools, prompts often appear to be the most important part of the process because prompts are the mechanism used to communicate with AI. However, prompts have a major limitation: they are temporary instructions rather than permanent requirements.

Consider a situation where two developers ask an AI assistant to create a secure login system. One developer might mention password complexity requirements, while another may forget to include them. Although both developers are requesting the same feature, the generated outputs may differ significantly because the prompts contain different assumptions.

This creates a problem for software teams. Prompts are difficult to review, difficult to version control, and difficult to validate against business requirements. Stakeholders cannot easily determine whether the generated solution truly matches their expectations.

A specification solves this problem by serving as a stable and shared reference point. It is reviewed before development begins, approved by stakeholders, and used throughout the project lifecycle. AI-generated code can be validated against it, test cases can be derived from it, and future enhancements can be planned around it.

Therefore, prompts tell the AI what to do at a particular moment, while specifications define what the product must do throughout its entire lifetime. This is why the specification becomes the source of truth in AI-assisted engineering.

4.1 Example

A simple prompt might look like:

> Build a secure login system.

This prompt does not answer several important questions:

- What authentication method should be used?
- What is the password policy?
- Should accounts be locked after repeated failures?
- How long should sessions remain active?

A specification provides these details explicitly:

```
Feature: User Login

Authentication:
Email and Password

Password Policy:
Minimum Length = 12 Characters

Account Lockout:
5 Failed Attempts
```

```
Session Timeout:
30 Minutes
```

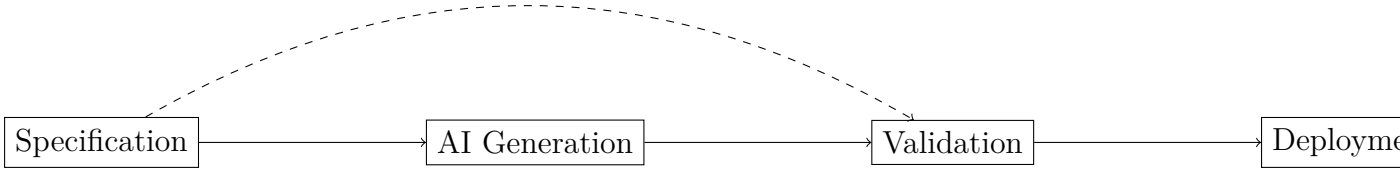
Because every requirement is clearly documented, all team members and AI systems work from the same understanding.

5 Prompt vs Specification

Prompt	Specification
Conversational	Structured
Temporary	Permanent
May be ambiguous	Clear and detailed
AI-focused	Team-focused
Hard to review	Easy to review
Hard to validate	Directly testable
Changes frequently	Version controlled

The key difference is that prompts help generate outputs, whereas specifications define the desired outcomes.

6 Architecture of AI-Assisted Engineering



The specification remains the central artifact throughout the lifecycle. Every output produced by AI or developers can be verified against it before deployment.

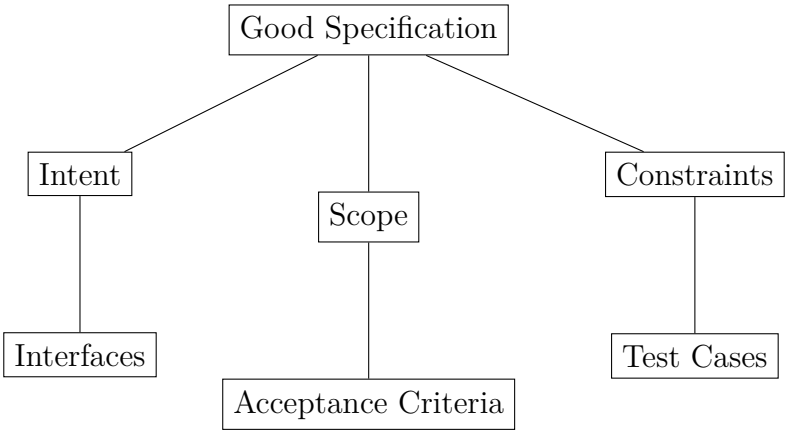
7 Anatomy of a Good Specification

A good specification tells the complete story of a feature. It explains why the feature is needed, what it should accomplish, what limitations exist, how it interacts with other systems, and how success will be measured.

Most high-quality specifications contain six essential components:

1. Intent
2. Scope
3. Constraints
4. Interfaces

5. Acceptance Criteria
6. Test Cases



8 Intent

Intent explains why a feature exists. It focuses on the business objective or user problem being solved. Without intent, developers may understand what needs to be built but may not understand why it matters.

For example, consider a book search feature. The intent is not simply to search a database. The real objective is to help users quickly discover books they are interested in, thereby improving user experience and increasing sales opportunities. By understanding the purpose behind the feature, development teams can make better implementation decisions.

9 Scope

Scope defines the boundaries of a feature. It clearly identifies what is included and what is excluded from the implementation.

For example, a book search feature might include searching by title, author, and category. However, voice search and image-based search might be intentionally excluded from the current release. Clearly defining scope prevents unnecessary expansion of requirements and helps the team focus on agreed objectives.

Included

- Search by title
- Search by author
- Search by category

Excluded

- Voice search
- Image search

10 Constraints

Constraints represent rules or limitations that the solution must follow. These may be technical, performance-related, security-related, or business-related.

For example, an organization may require the backend to be developed using FastAPI, the database to use PostgreSQL, and the response time to remain below two seconds. Such constraints ensure that the solution aligns with organizational standards and expectations.

- Backend: FastAPI
- Database: PostgreSQL
- JWT Authentication
- Response Time Less Than Two Seconds

11 Interfaces

Interfaces describe how different systems communicate with each other. They define the inputs and outputs that allow components to exchange information.

For example, a search service might expose the following API endpoint:

```
GET /books/search?q=python
```

The response may look like:

```
{
  "book_id": 201,
  "title": "Learning Python",
  "author": "Mark Lutz"
}
```

By documenting interfaces, teams ensure consistent communication between different parts of the system.

12 Acceptance Criteria

Acceptance criteria define the conditions that must be satisfied before a feature can be considered complete. These criteria are specific, measurable, and testable.

For the book search feature, acceptance criteria may include:

- Users can search by title.
- Users can search by author.
- Search is case-insensitive.
- Results are displayed within two seconds.
- A friendly message appears when no results are found.

Acceptance criteria provide a clear definition of success and help prevent misunderstandings between stakeholders and development teams.

13 Test Cases

Test cases verify whether the system satisfies the acceptance criteria. Each test case represents a specific scenario along with the expected outcome.

Test Case 1: Existing Book Search

Input:

```
Python
```

Expected Result:

```
Matching books are returned.
```

Test Case 2: Non-Existing Book Search

Input:

```
Unknown Book
```

Expected Result:

```
No books found.
```

Test Case 3: Case-Insensitive Search

Input:

```
pYtHoN
```

Expected Result:

```
Same results as "Python".
```

These test cases provide objective evidence that the implemented feature behaves as intended.

14 Conclusion

As AI becomes an integral part of software development, the need for clear and reliable requirements becomes even more important. Prompts are useful because they allow humans to communicate with AI systems, but they are not sufficient to guide a complex project over time. They can change from one interaction to another and often depend on the individual writing them.

Specifications provide a stronger foundation. They establish a shared understanding of requirements, reduce ambiguity, support testing and validation, and ensure consistency throughout the software lifecycle. For this reason, modern AI-assisted engineering treats the specification as the source of truth, while prompts serve merely as a means of interacting with AI systems. By placing specifications at the center of development, organizations can achieve greater reliability, maintainability, and overall software quality.